
SK hynix 사내 교육

AI Agent Guide

내 업무에 적용할 작은 AI 도구 만들기

강사 · NAND Implant기술 이상우 TL



하루 흐름



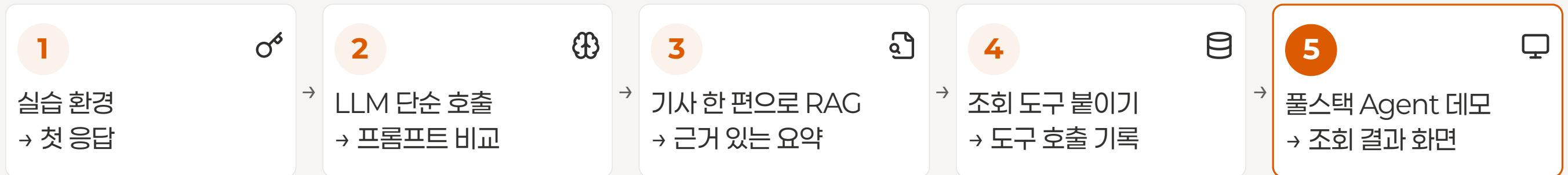
시작 09:00

점심 11:40-13:20

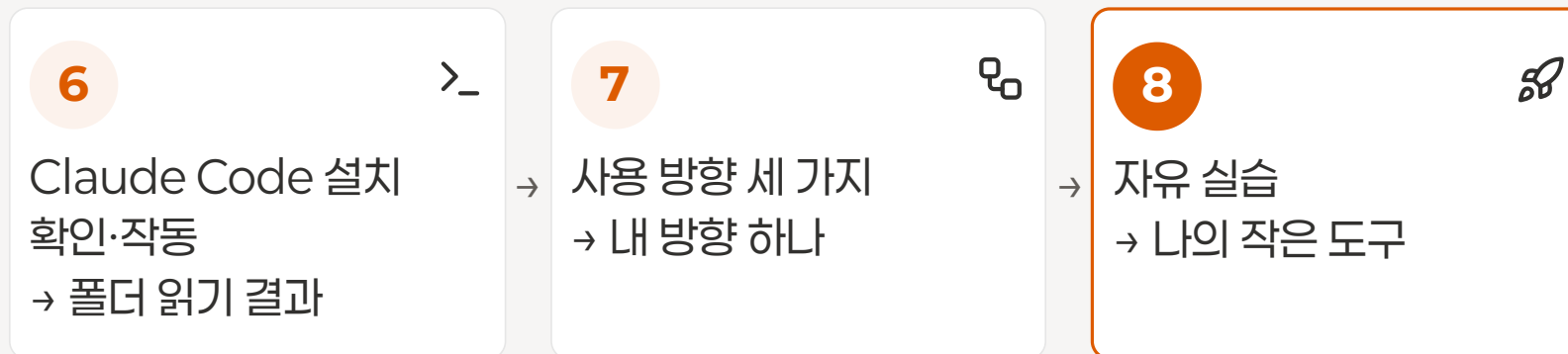
종료 17:30

휴식은 진행에 따라 안내합니다

오전 · 다섯 구간



오후 · 세 구간



→ 뒤는 구간의 산출물 · 다음 구간의 재료가 됩니다

강사 소개

비전공자의 출발점에서 만들며 배운 것을 소개합니다.

강사는 공정 업무를 하며 도구를 만들었고, 코드만큼 구조·데이터 검증·사용자 동선이 중요했습니다. 세 도구는 관련 개념 설명 바로 뒤에 사례로 다시 나옵니다.

NAND Implant기술 이상우 TL

기술 블로그

개인 기록



Part Finder

공개 Beta
부품 번호 후보 검증



ddokmo

개인 웹앱
사용 흐름 점검



Vault

Obsidian
원문과 판단 구분

[IMG-PLACEHOLDER · P-S02]

넣을 자료: 프로젝트 1개의 공개 화면

조건: 계정·사내 정보 가리기

대체: 확보 전엔 카드 설명으로 진행

AI를 잘 활용하는 공부법

만들고, 설명하고, 조건을 바꾸고, 검증하는 순서로 배웁니다.

네 단계는 구간마다 되풀이하는 공부 방법입니다. 먼저 만들되 기초는 결과를 설명하며 채우고, 실습 완료 조건에도 설명과 확인이 있습니다.



▶참고 영상 제목 "6년 걸릴 공부를 3일로 끝내는 법 — OpenAI 연구원이 인터뷰에서 직접 밝혔습니다" · 제목·썸네일 문구는 영상의 것이며 이 교육의 근거가 아님

- 1 만들기
- 2 설명하기
- 3 조건 바꾸기
- 4 검증하기

출처: YouTube · AgentOS 채널 · <https://www.youtube.com/watch?v=Kf1dYpnH-N4>

실습 환경의 세 층

코드는 내 PC에서 돌고, 계산은 사내 추론 서버에서 하며, 환경변수가 둘을 잇습니다.

설치는 사전 과제로 마쳤으니 오늘은 세 층이 이어지는지만 확인합니다. 코드에 OpenAI라는 이름이 있어도 요청은 환경변수에 적은 서버로 갑니다.



주소·모델명·키 대신 역할 이름으로 표시 · 한 층만 어긋나도 첫 응답이 오지 않습니다

출처: vLLM Online Serving 문서 · LangChain Models 문서

AI Agent Guide · SK hynix 사내 교육

요청 목적지를 정하는 두 줄 · 개념 예시

```
llm = ChatOpenAI(  
    model=MODEL,           # 모델 이름  
    base_url=BASE_URL,     # 서버 주소  
) # 키는 환경변수에서 읽음
```

코드 위치: 모델 연결 설정 — 실습 가이드 해당 장

☒ 첫 응답 = content

☐ 토큰 사용량도 함께

LLM의 응답 생성

LLM은 사실을 조회하지 않고 다음 토큰의 확률 분포를 계산합니다.

보통의 호출은 학습된 모델로 계산만 하므로, 질문을 더 보낸다고 모델이 재학습되지 않습니다. 한 번에 넣을 수 있는 입력·출력 길이의 한도를 컨텍스트 윈도우라고 합니다. 다음 실습에서 바꿀 temperature 값은 3단계의 선택에 영향을 줍니다.



고른 토큰을 입력 끝에 붙여 2단계로 돌아가기 — 답이 끝나거나 길이 한도에 닿을 때까지 반복

개념 예시 · 실제 토큰 분할은 모델마다 다름 · 확률 수치 없음 · 길이 한도 = 컨텍스트 윈도우

출처: Vaswani et al. 2017 · Ouyang et al. 2022 (지시 수행 학습)

Temperature와 출력 변화

temperature는 다양성을 조절할 뿐, 정확성 스위치가 아닙니다.

2단계에서 계산한 분포를 3단계에서 고를 때 얼마나 뾰족하게 볼지 정합니다. 바뀌는 것은 선택의 폭뿐입니다.

temperature 낮게



상위 후보에 몰림



- 결과가 비슷해지기 쉬움

temperature 높게



여러 후보로 퍼짐



- 표현이 자주 달라짐

두 값 공통



별도 확인

- 새 정보를 더하지 않음
- 낮아도 틀릴 수 있음
- 완전 동일도 보장 안 됨

같은 후보 5개의 분포 모양 · 개념 예시 · 수치 없음

출처: vLLM Reproducibility 문서 · LangChain Models 문서

메시지 역할과 프롬프트

system이 규칙을 정하고, user가 묻고, assistant가 답합니다.

모델에 보내는 것은 문장 하나가 아니라 역할이 붙은 메시지 목록입니다. 모델은 목록에 있는 것만 보므로, 앞의 답을 다시 넣지 않으면 이전 대화를 모릅니다.



system

규칙·말투·역할
한 줄이 답 전체를
바꿈



user

이번 질문
매번 새로 보냄



assistant

모델의 답
다음 요청에 다시
넣어야 기억

예시 몇 개를 함께 넣으면 **답의 모양이 예시를 따릅니다**

역할이 붙은 메시지 목록 · 개념 예시

```
messages = [  
  ("system", "정중하게 답합니다"),    # 규칙  
  ("user", "첫 질문"),  
  ("assistant", "첫 답"),              # 앞 답을 다시 넣음  
  ("user", "두 번째 질문"),  
]
```

코드 위치: 프롬프트 구성부 · 긍정봇 — 실습 가이드 해당 장

출처: LangChain Messages 문서 · OpenAI Chat Completions 가이드

실습 1 · 첫 호출과 프롬프트

코드 위치: 모델 연결 설정 · 모델 호출부 · 긍정봇 — 실습 가이드 해당 장

관찰할 것 같은 질문에서 system 한 줄과 temperature가 각각 무엇을 바꾸는지, 긍정봇이 규칙 없이 → system만 → 예시 추가로 어떻게 달라지는지 봅니다. 두 번째 말에 앞 대화를 기억하는지, 최신 자료 답에 근거가 있는지도 표시합니다.



① 환경 · 세 층

② 역할 · system

③ temperature

④ 역할 · 예시

⑤ 환각

완료 조건

바꾼 것 하나와 달라진 것
하나를 한 문장으로 설명합니다

막히면 접속이 안 되면 준비된 출력으로 진행합니다

[IMG-PLACEHOLDER · P-S08]

넣을 자료: 긍정봇 3단 비교 캡처

조건: 세 답을 한 화면에 · 주소·키 가리기

대체: 확보 전에는 왼쪽 안내로 진행

단순 호출의 한계와 RAG

말투와 다양성은 바꿀 수 있지만, 모델이 모르는 자료는 답하지 못하므로 다음은 자료를 붙이는 RAG입니다.

실습 1에서 system 한 줄과 temperature만으로 답의 모양은 바뀌었지만, 최신 자료를 물었을 때는 그럴듯한 문장이 근거 없이 나올 수 있었습니다. 다음 실습은 모델이 모르는 기사 본문을 질문과 함께 전달해 그 빈자리를 채웁니다.

① 지금 되는 것



실습 1 · 첫 호출

- system 한 줄로 말투 바꾸기
- temperature로 다양성 조절
- 규칙·예시로 답 모양 정하기

② 아직 안 되는 것



모델만으로는 한계

- 학습 이후의 최신 자료
- 사내 문서·우리 부서 자료
- 출처를 대며 답하기

③ 그래서 다음



실습 2 · 기사로 RAG

- 기사 본문을 찾아 붙이기
- 답을 본문과 대조
- 없으면 없다고 하기

실습 1 · 첫 호출



실습 2 · 기사 한 편으로 RAG

모르는 자료는 찾아서 붙입니다

출처: Lewis et al. 2020

외부 자료가 필요한 질문

그럴듯함과 근거 있음은 다릅니다.

모델은 학습 이후 정보와 사내 자료를 모르고 출처도 보장하지 못합니다. 이렇게 그럴듯하지만 근거 없는 답을 환각이라 부릅니다. 근거를 찾아 질문과 함께 전달하면 이번 답변에 쓰게 할 수 있지만, 검색·자료 품질 확인은 여전히 필요합니다.

학습 이후 정보 없음

사내 자료 없음

출처 보장 못 함

● 모델만 사용



- 문장이 자연스러워도 근거는 따로 확인 · 환각
- 출처를 물어도 지어낼 수 있음

● 근거 전달



- 답변 문장을 기사와 대조할 수 있음
- 검색·자료 품질은 여전히 따로 확인

출처: Lewis et al. 2020 · LangChain Retrieval 가이드

RAG의 기본 구조

미리 준비하는 인덱싱과 질문할 때의 검색·답변, 두 시점이 있습니다.

RAG는 질문에 맞는 자료 조각을 찾아 질문과 함께 모델에 넘기는 방식입니다. 인덱싱은 문서를 나누고 숫자로 바꿔 저장해 두는 과정이고, 검색 방법은 여러 가지가 있습니다. 모델을 학습시키지 않으므로 자료를 기억하지 않습니다.



개념 예시  **조각 1** **겹침** **조각 2** **겹침** **조각 3**

위 흐름은 일반 구조 · 분할 크기·겹침: 작으면 문맥 손실, 크면 잡음

코드 위치: 자료 준비(수집·분할) 부분 · 검색 부분 — 실습 가이드 해당 장

출처: Lewis et al. 2020 · LangChain Retrieval 가이드

벡터 저장소가 필요한 이유

글자가 같은 것이 아니라 뜻이 가까운 것을 빨리 찾으려고 합니다.

키워드 검색은 같은 낱말이 있어야 찾지만, 뜻을 숫자 묶음(벡터)으로 바꿔 두면 표현이 달라도 가까운 조각을 찾을 가능성이 커집니다. 조각이 수천·수만 개가 되면 하나하나 비교할 수 없어, 벡터를 저장하고 가까운 것을 빠르게 찾아 주는 전용 저장소를 둡니다. 오늘 실습은 기사 한 편이라 조각이 몇 개뿐이지만, 같은 구조를 씁니다.

개념 예시

키워드 검색



같은 낱말만 찾음

- 질문: "장비 고장"
- 조각: "설비 이상"
- 결과: 못 찾음

뜻으로 찾기



뜻이 가까우면 찾음

- 질문: "장비 고장"
- 조각: "설비 이상"
- 결과: 찾음



조각들



뜻을 숫자로



벡터 저장소



질문



가까운 것 몇 개

윗줄 · 오늘 실습에서 손으로 하는 저장

아랫줄 · 저장해 둔 조각을 나중에 쓰는 방식

코드 위치: 자료 준비(분할·임베딩·저장) 부분 — 실습 가이드 해당 장

출처: Lewis et al. 2020 · LangChain Vector stores 문서

FAISS · Chroma · Milvus

셋 다 벡터를 저장하고 가까운 것을 찾지만, 어디서 얼마나 크게 돌리느냐가 다릅니다.

FAISS는 검색 라이브러리로 내 프로그램 안에서 바로 쓰고 파일로 저장할 수 있어 오늘 실습에 씁니다. Chroma는 설치가 간단한 벡터 DB로 내 PC 프로젝트에 붙이기 쉽고, Milvus는 서버로 띄워 여러 사람·많은 자료를 다루는 규모용입니다. 우열이 아니라 규모와 운영 방식의 차이입니다.

라이브러리 · 서버 없음 · 오늘 실습 FAISS - 프로그램 안에서 바로 검색 - 결과(인덱스)를 파일로 저장	가벼운 DB · 작은 팀 Chroma - 설치 간단, 로컬에 저장 - 출처 정보로 걸러 찾기	서버형 DB · 사내 공용 규모 Milvus - 여러 사용자·대용량 - 분산·운영 기능
공통: 벡터 저장 + 가까운 것 찾기 · 다른 점: 규모·운영		

개념 예시 · 실습 코드의 저장소 이름은 실습 가이드 해당 장
출처: FAISS 공식 문서(GitHub wiki) · Chroma 공식 문서 · Milvus 공식 문서

나누고, 숫자로 바꾸고, 가까운 것 찾기

긴 자료는 조각으로 나누고, 조각을 뜻이 담긴 숫자로 바꿔 두면 가까운 조각을 찾을 수 있습니다.

한 번에 넣을 수 있는 길이에 한도가 있고, 자료가 크면 질문과 관계없는 부분까지 섞이므로 나눕니다. 나눈 조각을 숫자 묶음으로 바꿔 두면, 질문과 뜻이 가까운 조각만 골라 모델에 넘길 수 있습니다.



개념 예시

반도체 공정

↔

웨이퍼 제조

숫자가 가까움 ↔

맛있는 김치

숫자가 멀

조각 수는 조각 크기에 따라 달라집니다 · 거리 계산 방식은 실습 가이드 해당 장

출처: Lewis et al. 2020 · LangChain Retrieval 가이드(Text splitters · Embeddings)

실습 2 · 기사 한 편으로 RAG 만들기

코드 위치: 본문 수집 부분 · 나누기와 저장 부분 · 본문을 붙여 요약하는 호출부 — 실습 가이드 해당 장

관찰할 것 조각이 몇 개 나오는지(두 개 안팎)와 chunk_size를 바꾸면 몇 개가 되는지 셉니다. 요약의 한 문장이 본문 어디에 있는지, 기사에 없는 것을 물으면 없다고 하는지 확인합니다.



- ① 찾아 읽기
- ② 나누기 · 임베딩
- ③ 근거 대조
- ④ 없으면 없음

완료 조건

요약의 한 문장을 뒷받침하는 본문 문장을 찾습니다

막히면 기사 접속이 안 되면 준비된 저장 본문으로 진행합니다

[IMG-PLACEHOLDER · P-S13]

넣을 자료: 조각 수 출력 · 요약 결과와 본문을 나란히 둔 화면

조건: 주소·키·모델명 가리기 · 1600×900 이상

대체: 확보 전에는 왼쪽 안내로 진행

출처: LangChain Retrieval 가이드

답변과 검색 근거 확인

틀렸다면 검색이 틀렸는지, 근거를 잘못 설명했는지부터 나눕니다.

RAG 품질은 검색(관련 조각을 가져왔나)과 답변(근거에 충실했나)으로 나뉘 봅니다. 검색 문제면 분할·검색 방식을, 답변 문제면 프롬프트 제약을 고칩니다. 정확한 집계 질문은 다음 구간의 도구(Tool)로 넘깁니다.

문장마다 대조



근거에 있나요?

- 근거에 있음: 문단에 같은 내용·날짜
- 근거 밖: 문단에 없는 추측·설명

고칠 곳 찾기



구분	답변 맞춤	답변 틀림
검색 맞춤	● 통과	프롬프트 제약
검색 틀림	분할·검색	둘 다 점검

출처: LangChain Retrieval 가이드 · Lewis et al. 2020

RAG의 한계와 도구

근거 있는 답은 되지만 정확한 숫자와 지금 상태는 못 내므로, 다음은 코드가 실행하는 도구입니다.

실습 2에서 본문을 붙이자 답을 본문과 대조할 수 있었고, 없는 내용은 없다고 답하게 만들었습니다. 그러나 건수를 세거나 지금 장비 상태를 보는 일은 문서에 없으므로, 다음 실습은 조회 함수를 도구로 붙여 모델이 부르고 코드가 실행합니다.

① 지금 되는 것



실습 2 · RAG

- 본문을 붙여 근거 있는 요약
- 답 문장을 본문과 대조
- 없으면 없다고 답하기

② 아직 안 되는 것



문서만으로는 한계

- 정확한 건수·집계
- 지금 이 순간의 장비 상태
- 조회·실행 같은 행동

③ 그래서 다음



실습 3 · 조회 도구

- 정해진 조회문으로 테이블 읽기
- 같은 함수를 도구로 등록
- 부를지는 모델, 실행은 코드

실습 2 · RAG



실습 3 · 조회 함수를 도구로

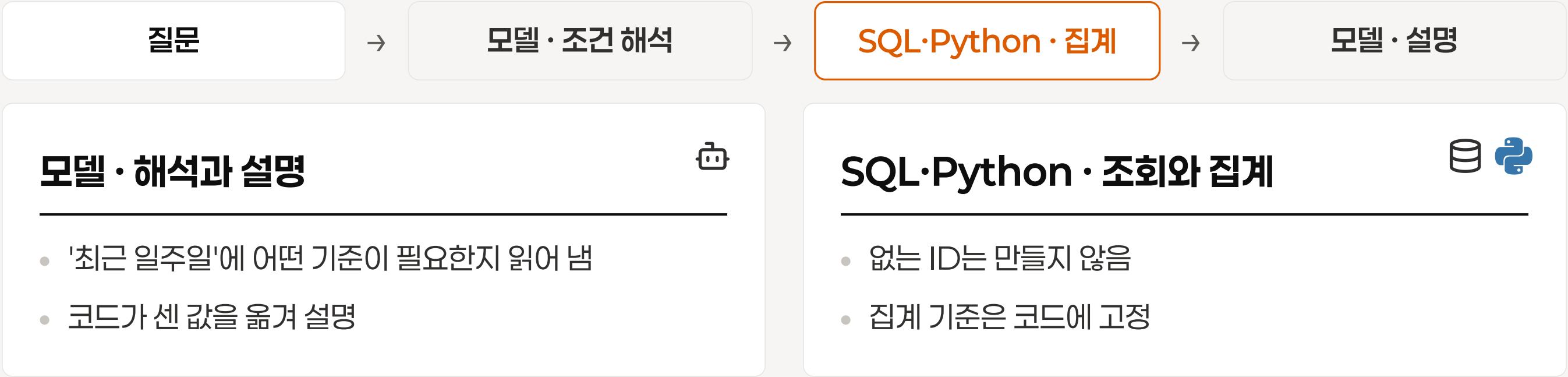
도구가 붙어야 Agent로 갈 수 있습니다

출처: LangChain Tools 가이드

정확한 조회와 계산

숫자와 ID는 코드가 맞고, 모델은 해석하고 설명합니다.

'최근 일주일 장비별 건수'는 날짜 기준·단위·중복 처리를 정해야 답이 하나로 나옵니다. 집계 기준은 코드에 고정하고, 정답을 아는 작은 데이터로 먼저 확인합니다.

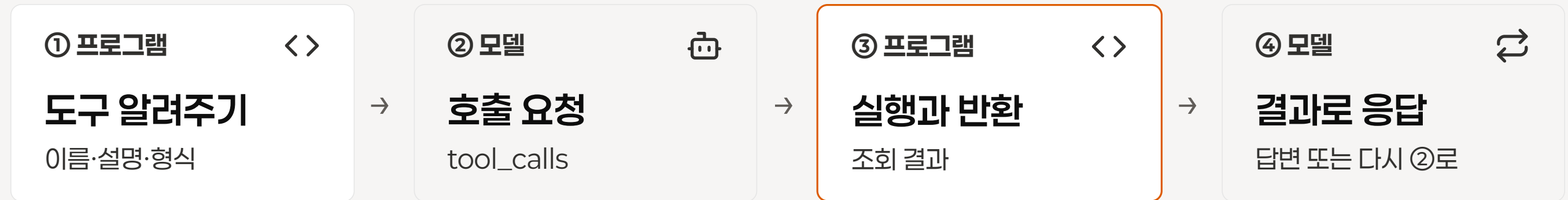


출처: LangChain SQL Agent 가이드 · 흐름도는 개념 예시 · 오늘 실습에서 모델이 정하는 것은 부를지 말지뿐

도구(Tool) 호출 순서

모델은 도구 호출을 요청하고, 실행은 프로그램이 합니다.

프로그램이 도구의 이름·설명·입력 형식을 알려 주면 모델은 부를 도구 이름을 담은 호출 요청을 돌려줍니다. @tool은 docstring이 곧 도구 설명이라 모호하면 잘못 고릅니다. 이 도구는 인자가 없어 모델은 '부르지 말지'만 정합니다.



④에서 도구 호출이 더 없으면 답하고 끝냅니다 · 더 있으면 ②로 돌아가 반복합니다

```
@tool # 교육용 가상 함수 · 인자 없음
def equipment_status() -> list:
    """장비별 현재 위치와 진행 공정"""
```



출처: LangChain Tools · OpenAI Function calling 가이드 · 코드는 개념 예시

실습 3 · 조회 함수를 도구로

코드 위치: 사내 데이터 조회 라이브러리 호출부 · @tool 연결 부분 — 실습 가이드 해당 장

관찰할 것 같은 함수가 도구가 되기 전과 후에 무엇이 달라지는지 봅니다. 도구가 필요 없는 질문에서 도구 호출 기록이 비는지 확인합니다. 이 도구는 정해진 쿼리라 인자가 없습니다.



- ① 조회 함수
- ② 숫자는 코드가
- ③ **tool_calls**
- ④ 부를지 말지

완료 조건

정답을 아는 질문과 도구 없이
답할 질문을 구분해 설명합니다

막히면 접속이 안 되면 준비된 조회 결과로 진행합니다

실행 환경에서 막는 네 층

읽기 전용	허용 테이블
쿼리 검사	사람 승인

출처: LangChain Tools 가이드 · LangChain SQL 안전 지침(SQL Agent 가이드 내)

스크립트·Workflow·Agent

문제에 필요한 만큼의 자율성만 줍니다.

Workflow는 코드가 순서를 정해 모델·도구를 부르고, Agent는 모델이 다음 행동을 고릅니다. Agent는 호출이 늘어 시간·사용량이 커지기 쉽고 결과를 예측하기 어려워질 수 있습니다.

규칙이 명확



스크립트

규칙 → 결과

모델 없이 코드가 끝까지

순서가 고정



Workflow

검색 → LLM → 답변

순서는 코드가 정함

상황에 따라



Agent

LLM ↔ 도구 반복

다음 행동은 모델이 정함

세 방식은 우열이 아니라 필요한 자율성의 차이 · 개념 예시

출처: Anthropic, Building effective agents

AI Agent Guide · SK hynix 사내 교육

도구의 한계와 화면

도구까지 붙인 함수는 코드를 아는 사람만 쓰므로, 다음은 누구나 쓰는 화면입니다.

실습 3에서 질문을 넣으면 모델이 조회 도구를 부르고 결과를 문장으로 설명하는 것까지 확인했습니다. 그러나 이 함수는 코드를 열어 실행할 줄 아는 사람만 쓸 수 있으므로, 다음 실습은 입력 칸과 결과 화면을 붙여 코드를 모르는 동료도 쓰게 합니다.

① 지금 되는 것



실습 3 · 도구

- 질문하면 모델이 도구를 부름
- 도구가 필요한 질문 구분
- 결과를 문장으로 설명

② 아직 안 되는 것



코드만으로는 한계

- 코드를 모르는 동료가 쓰기
- 입력 칸·결과 화면
- 결과 없음 같은 안내

③ 그래서 다음



실습 4 · 풀스택 Agent 데모

- Streamlit 화면에 연결
- 장비 ID 입력 → 위치·진행 공정
- 입력 → 함수 → 화면 연결

실습 3 · 도구



실습 4 · 풀스택 Agent 데모

오후에는 이 앱을 Claude Code가 읽습니다

출처: Streamlit Architecture 문서

웹앱의 화면·처리·데이터

Streamlit 앱도 화면·처리·데이터로 나눠 읽습니다.

입력을 받고, 데이터를 조회해 처리한 뒤 화면에 다시 보여 주는 전체 흐름을 흔히 풀스택이라 부릅니다. 화면·서버 기술을 다 배우기 전에, 바꿀 부분이 어느 층인지부터 알아봅니다.

개념 예시 · 함수 이름은 가상 · 실제 실습 코드와 다를 수 있음



```
q = st.text_input("질문")    # 화면: 입력
if q:
    rows = search(q)         # 데이터: 조회
    ans = answer(q, rows)    # 처리: 규칙·모델
    st.write(ans)            # 화면: 결과
```

[IMG-PLACEHOLDER · P-S29]

넣을 자료: 오전 Streamlit 앱 화면 — 입력 칸·결과 영역을 왼쪽 코드 줄과 짝지어 표시

조건: 1600×900 이상 · 실행일 표기 ·
모델명·주소·키 가리기

대체: 확보 전에는 왼쪽 코드 예시로 설명

출처: Streamlit Architecture 문서 · Streamlit API 문서(st.text_input, st.write)

실습 4 · Streamlit 기반 데모형 풀스택 Agent

🚩순서: 실행 → 장비 ID 입력 → 연결 짚기 · 코드 위치: 화면 앱 파일 — 실습 가이드 해당 장

관찰할 것 입력한 장비 ID가 어떤 경로로 조회 함수와 모델에 전달되고, 결과가 어느 코드에서 화면에 그려지는지 짚습니다. 세 영역이 앞 장의 화면·처리·데이터 중 어디인지 봅니다.

- ① 앱 실행 — 화면 코드를 실행해 브라우저에서 엽니다
- ② 장비 ID 입력 — 위치와 진행 공정이 나오는지 확인합니다
- ③ 연결 짚기 — 입력 → 조회 함수 → 화면 결과의 줄을 찾습니다

① 화면

② 데이터

③ 처리

완료 조건

입력 → 함수 → 화면 결과의 연결을 설명합니다

막히면 옆 사람 화면으로 함께 확인하고, 계속 막히면
강사 시연으로 진행합니다

[IMG-PLACEHOLDER · P-S20]

넣을 자료: 장비 조회 챗봇 화면 · 세 영역에 ①–③ 번호

조건: 주소·키·모델명·실제 장비 ID 가리기 · 1600×900 이상

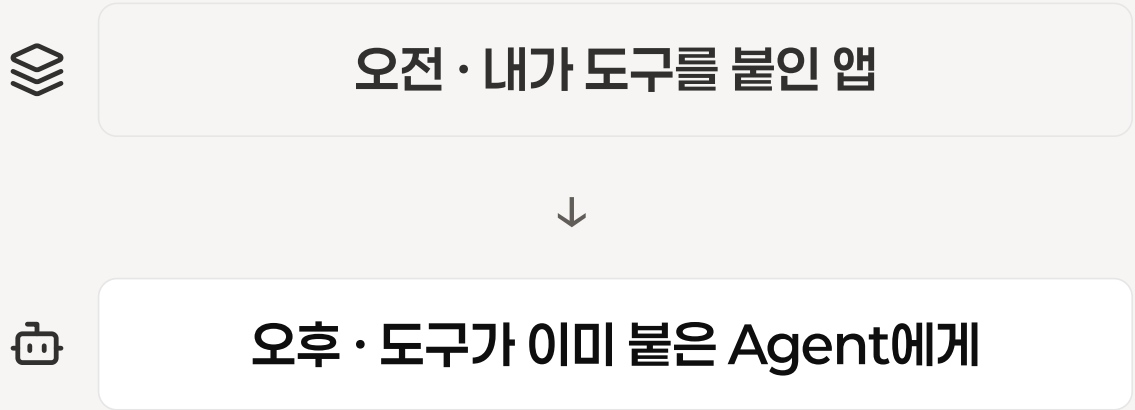
대체: 확보 전에는 왼쪽 ①–③으로 진행

출처: Streamlit Architecture 문서

오전 실습의 전체 구조

오전에 모델 둘레에 붙인 것들이 곧 하네스이고, 오후엔 이 앱을 Agent에게 맡깁니다.

화면·모델과 도구·데이터 세 층을 오늘 손으로 이었습니다. 결과 없음 안내 하나를 더하려 해도 세 층을 다시 손으로 만져야 합니다. 그래서 오후에는 도구가 이미 붙은 Agent에게 이 구조를 읽히고, 바꿀 범위를 정하게 합니다.



주황 테두리: 오후 시연과 선택 과제의 대상(예: 결과 없음 안내)

출처: Streamlit Architecture 문서 · Anthropic, Building effective agents

점심시간

오후는 Claude Code 실행 확인부터 시작합니다.

11:40

점심 시작



사전 설치한 Claude Code



오전 실습 폴더



아침에 적은 반복 업무 하나

- 준비물
- Claude Code
- 오전 실습 폴더
- 반복 업무 하나

Coding Agent의 작동 방식

모델에 도구를 붙인 점은 같고, 대상이 프로젝트로 넓어지며 관찰·판단·행동을 반복합니다.

Claude Code는 모델에 파일·명령 도구를 붙여 관찰→판단→행동을 반복하는 Coding Agent입니다. 오후에는 이 흐름으로 오전에 만든 장비 조회 챗봇을 읽고, 시연과 선택 과제의 대상으로 씁니다. 다른 도구보다 항상 낫다는 뜻은 아닙니다.

오전에 만든 장비 조회 챗봇



- 대상: 장비 ID 질문
- 도구: 우리가 붙인 조회 도구
- 결과: 위치·진행 공정 답변

* Claude Code 개념 예시

관찰
파일·결과



판단
모델이 선택



행동
도구 실행

대상: 프로젝트 폴더 · 결과를 보고 다시 반복

출처: Claude Code How Claude Code works · Building effective agents · Kaggle Agents 백서

로컬 실행과 접근 범위

실행 위치와 모델이 처리하는 위치는 다릅니다.

파일 읽기·수정과 명령은 내 PC에서 실행되지만, 요청과 읽은 파일 내용은 모델 경로로 전송됩니다. 그래서 허용된 경로와 보낼 자료 범위를 먼저 확인합니다. 화면 조작·사내 시스템은 별도 도구·권한이 필요할 수 있습니다.



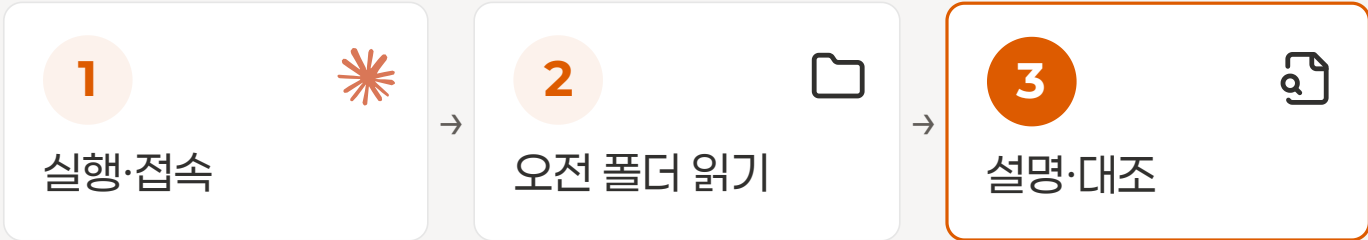
- 파일·명령은 내 PC에서 실행
- 모델 처리는 설정된 경로에서
- 할 수 있는 일은 도구·권한 범위

출처: Claude Code 공식 문서 How Claude Code works · Permissions · Data usage

실습 5 · 설치 확인·접속·폴더 읽기

코드 위치: 오전 실습 폴더(장비 조회 챗봇) — 실습 가이드 해당 장

관찰할 것 각 파일의 역할과 앱 실행 방법을 설명받은 뒤 실제 파일을 열어 대조합니다. 앞서 예상한 것 중 파일 읽기가 되는지 확인하고, Python 실행은 승인 요청이 뜰 때 봅니다.



- ① 실행 환경
- ② 관찰
- ③ 판단은 내가

완료 조건
구조 설명을 실제 파일과 대조합니다

막히면 짝·강사 시연 화면으로 진행합니다 · 우회 접속은 하지 않습니다

[IMG-PLACEHOLDER · P-S26]

넣을 자료: 오전 실습 폴더 구조를 설명한 Claude Code 화면

조건: 모델명·계정·주소·키 가리기 · 실행일 표기

대체: 확보 전에는 강사 시연 화면으로 대신

막힌 단계에 손을 들어 알려 주세요

- 

설치

접속

폴더 열기

파일 읽기

실행 방법

출처: Claude Code 공식 문서 How Claude Code works · Best practices

Claude Code 사용 방향 세 가지

같은 도구를 세 방향으로 씁니다.

새 용어는 없습니다. 방향마다 오늘 볼 것 하나와 오전에 본 원칙 하나를 짚지어 이어서 살펴봅니다.

방향 ①



Agent 그 자체

- 로컬 파일 데이터 분석
- 예: CSV 집계·요약

오늘 볼 것: 계산은 코드가, 설명은 모델이

방향 ②



Coding Agent

- 스크립트·작은 앱 만들기
- 예: 엑셀 비교 스크립트, 부품 검색 도구

오늘 볼 것: 완료 조건을 요청문에 넣기

방향 ③



지식 자산화

- Obsidian 노트와 함께 나만의 Agent
- 예: 새 개념을 넣고 다시 묻기

오늘 볼 것: AI가 만든 정리는 검토 전으로

출처: Claude Code 공식 문서 Overview

AI Agent Guide · SK hynix 사내 교육

방향 ① · 로컬 파일 데이터 분석

계산은 코드가, 모델은 요청과 설명을 맡습니다.

질문을 받은 Claude Code가 Python 스크립트를 쓰고 실행해 표·그래프를 만든 뒤 결과를 설명합니다. 오전 조회 도구에서 숫자는 코드가 세게 했던 것과 같은 원칙입니다.



오전 원칙 재사용 · 숫자와 ID는 코드가, 해석과 설명은 모델이

[IMG-PLACEHOLDER · P-C02]
넣을 자료: CSV 집계 시연 화면(질문·실행·결과 표)
조건: 가상 CSV 사용 · 모델명·경로·계정 가리기
대체: 확보 전에는 강사 시연 화면으로 대신

출처: Claude Code 공식 문서 Best practices

작업 요청과 완료 조건

입력 예와 기대 결과가 곧 완료 조건입니다.

여섯 칸을 채우면 작업 방향과 확인 방법이 함께 정해집니다.

● "챗봇 개선해줘" — 무엇을·어디까지·언제 끝인지 없음

목표 없는 장비 ID일 때 안내 표시	입력 없는 장비 ID로 질문	출력 "해당 장비 없음" 안내 문구
범위 화면·처리만 수정, 데이터 그대로	제약 새 패키지 설치 없이	완료 조건 없는 ID → 안내, 기존 질문 그대로

● 오전 장비 조회 챗봇에 결과 없음 안내 붙이기 · 완료 조건이 있는 요청문 6칸 · 개념 예시

출처: Claude Code 공식 문서 Best practices

탐색·계획·구현·검증

바로 고치게 하지 말고, 읽고 계획하게 한 뒤 범위 안에서만 고칩니다.

여러 파일을 바꾸거나 방법이 불확실할 때는 계획을 먼저 세우고, 검증에서 문제가 보이면 탐색으로 돌아갑니다. 방향 ② 시연을 이 네 단계로 끊어 보겠습니다.



[IMG-PLACEHOLDER · P-S28] 넣을 자료: 방향 ② 시연(오전 챗봇 결과 없음 안내) 네 단계 화면 · 조건: 모델명·경로·계정 가리기 · 대체: 확보 전에는 강사 실시간 시연

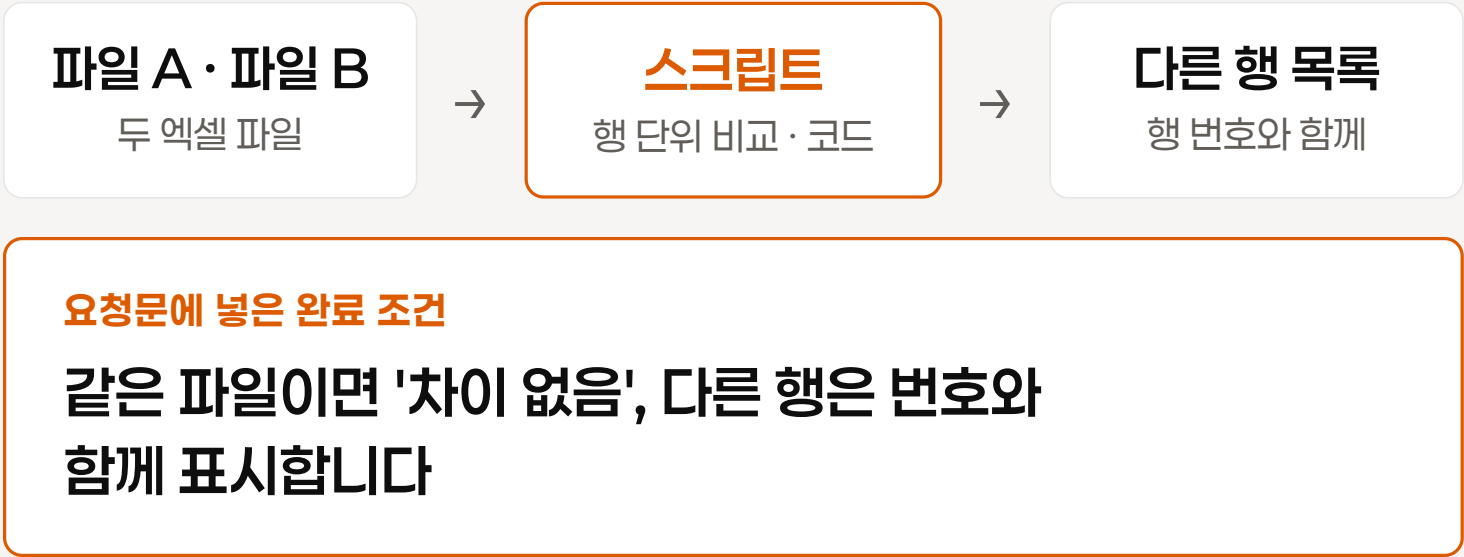
출처: Claude Code 공식 문서 Best practices

방향 ② · 스크립트와 작은 앱 만들기

완료 조건에 '없으면 없다고 답한다'를 넣어야 쓸 수 있는 도구가 됩니다.

엑셀 비교 스크립트든 부품 검색 도구든 요청문의 완료 조건이 결과의 모양을 정합니다. 오른쪽은 강사가 만든 부품 검색 도구의 공개 Beta 설계에서 넣은 완료 조건입니다.

🔗 예 · 엑셀 비교 스크립트



 공개 Beta 개념 예시

요청: 제조사 A의 로봇 블레이드

Part Number	종류	제조사	판정
PN-A-2001	O-ring	A	● 종류 불일치
PN-B-1001	로봇 블레이드	B	● 제조사 불일치
결과: 조건에 맞는 후보 없음			

완료 조건: 후보 번호만 · 조건 모두 일치 · 없으면 '없음' · 가상 데이터

출처: Claude Code 공식 문서 Best practices

프로젝트 지침과 작업 기록

긴 대화는 영구 기억이 아닙니다 — 다음 작업에 필요한 사실은 파일에 남깁니다.

CLAUDE.md는 모델이 읽는 지침일 뿐 권한을 막는 보안 장치가 아닙니다. 오전 긍정봇에서 보듯이 모델은 이번에 보낸 목록만 보므로, 새 대화를 열거나 긴 대화가 요약된 뒤에도 필요한 사실이 남도록 합니다.

CLAUDE.md · 프로젝트 지침 예시	작업 기록 · 예시 양식
## 저장소 성격 ## 실행 명령 ## 주의점 - 숫자는 코드로 계산	결정: 무엇을 하기로 했나 이유: 왜 그렇게 정했나 검증: 무엇으로 확인했나 다음: 남은 일과 다음 행동

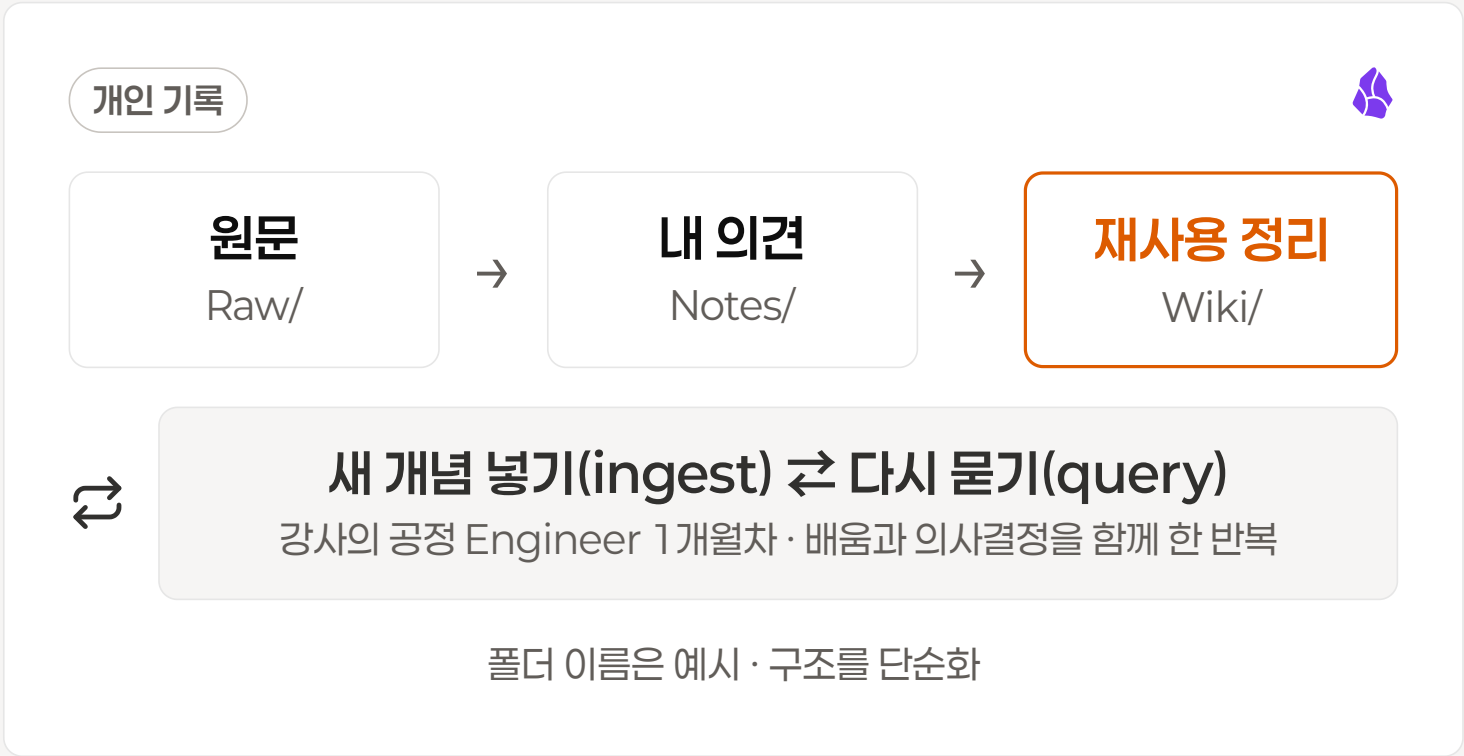
오전에 본 컨텍스트 윈도우 현재 대화 + 읽은 파일 + 도구 출력 · 한도가 있고 길면 요약됩니다

출처: Claude Code 공식 문서 Memory · Context window

방향 ③ · Obsidian 지식 자산화와 나만의 Agent

자료를 쌓는 것과 내 판단 기준이 생기는 것은 다릅니다.

원문, 내 의견, 정리를 나눠 두고, 새 개념을 수시로 넣고(ingest) 다시 묻는(query) 반복이 배움과 의사결정을 함께 합니다.



[IMG-PLACEHOLDER · P-C04]

넣을 자료: 공개 가능한 Obsidian 노트 한 장

조건: 개인·사내 정보 가림 · 1600×900 이상

대체: 확보 전엔 왼쪽 그림으로 진행

Obsidian 파일 = Memory

CLAUDE.md = Context

Skill = 절차 파일

검토 전 AI가 만든 정리는 검토 전으로 둡니다

출처: Claude Code 공식 문서 Memory

자유 실습 주제 선택

결과 하나와 완료 조건 두 가지를 먼저 정합니다.

본인 업무를 골라도 됩니다. 자료는 가상·공개 가능한 것만, 되돌리기 어려운 작업은 뺍니다. 주제가 떠오르지 않으면 다음 장의 선택 과제부터 시작합니다.

업무 데이터 · 방향 ①



CSV 집계

처음이면 추천

- 결과: 장비별 건수표
- 확인: 정답 입력 일치
- 예외: 빈 파일 안내

배울 점: 계산은 코드, 기준은 사람

오전 연계 · 방향 ②



오전 챗봇 개선

처음이면 추천

- 결과: 기능 하나 더
- 확인: 기존 입력 유지
- 예외: 없는 장비 ID 안내

배울 점: 코드 이해·범위·기존 기능 확인

개인 지식 · 방향 ③



지식 정리

- 결과: 출처 연결 노트
- 확인: 근거 문장 · 사실·의견 구분
- 예외: 자료에 없으면 '없음'

배울 점: 출처·검토

선택 과제 · 오전 챗봇에 결과 없음 안내 붙이기

자유 주제가 정해지지 않은 분을 위한 과제 · 강제 아님 · 코드 위치: 화면·처리 부분 — 실습 가이드 해당 장

관찰할 것 파일 수정·명령 실행 승인 요청이 뜨면 무엇을 하려는지 읽고 나서 승인합니다. 검증은 실행 전에 결과를 예측한 뒤 다른 입력으로 한 번 더 확인합니다.



① 읽기

② 완료 조건

③ 범위 안에서

④ 예측 먼저

완료 조건

기존 질문은 그대로, 없는 장비 ID에는 안내가 나옵니다

승인 요청 · 개념 예시(실제 화면과 다름)



수정 요청 · 대상: 화면·처리 부분
허용할까요? 예 / 아니오

판단 기준 완료했다는 말이 아니라 바뀐 줄과 실행
결과로 판단합니다

막히면 짝 화면으로 같은 단계를 진행하고, 앱의 모델 호출이
막히면 준비된 결과로 확인합니다

출처: Claude Code 공식 문서 Best practices · Permissions

자유 실습 요청문

요청 초안 작성

관찰할 것 기능을 여러 개 적지 않고 가장 먼저 쓸 결과 하나를 적습니다. 완료 조건에는 정상 입력과 빈·잘못된 입력의 기대 결과를 함께 씁니다.



목표
무엇을 만들까요 · 결과 하나



입력
어떤 자료·파일을 쓰나요



출력
받을 결과의 형식



범위
이번에 바꿀 곳



제약
환경·자료 조건



완료 조건
정상·예외 입력의 기대 결과

이번 실습 완료 조건
여섯 칸을 채운 요청 초안 한 개

막히면
주제가 정해지지 않았으면 앞 장의 선택 과제(결과 없음 안내)나 CSV 과제(장비별 건수표)로 시작합니다

출처: Claude Code 공식 문서 Best practices

자유 실습 1단계 · 목표와 계획

실습 가이드 해당 장

관찰할 것 요청문의 결과물이 하나로 줄었는지, 완료 조건이 입력과 기대 결과로 적혔는지 봅니다. 사내 DB 쓰기·외부 전송처럼 되돌리기 어려운 작업은 범위 밖입니다.



- 결과물 하나로 범위 줄이기
- 탐색 후 작업 순서 계획

완료 조건

결과물 하나와 완료 조건 둘 이상

목표·범위 → 탐색·계획

막히면 실행 환경 문제는 짝 실습으로, 주제가 크면 선택 과제나 준비된 CSV 과제로 바꿉니다

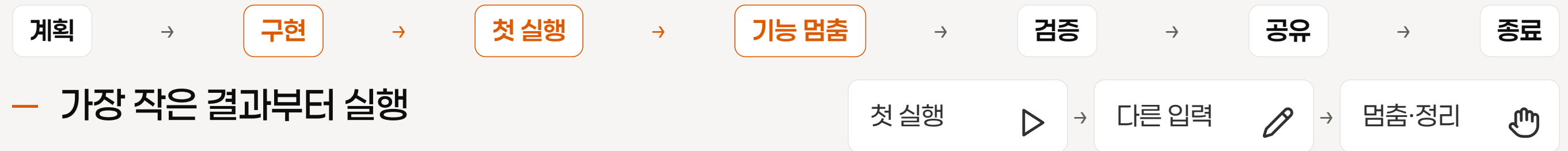
요청문 범위 완료 조건

출처: Claude Code 공식 문서 Best practices

자유 실습 2단계 · 구현

실습 가이드 해당 장

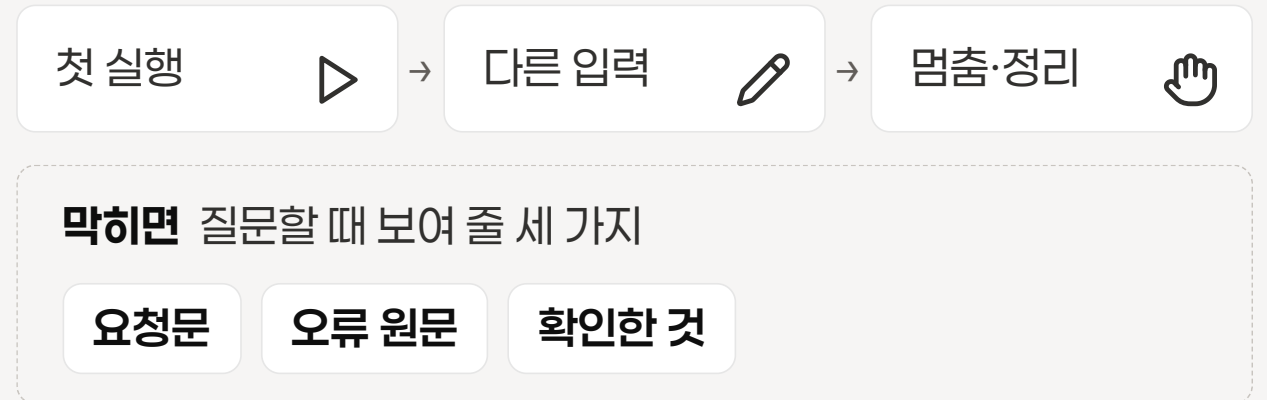
관찰할 것 가장 작은 결과로 첫 실행이 먼저 되는지, 다른 입력에서도 기대한 결과가 나오는지 봅니다. 안 되면 기능을 하나 줄이고, 막힌 단계와 오류 원문을 기록합니다.



- 가장 작은 결과부터 실행
- 실행할 때마다 결과 확인

완료 조건

검증 전에 실행되는 결과 하나가 있습니다



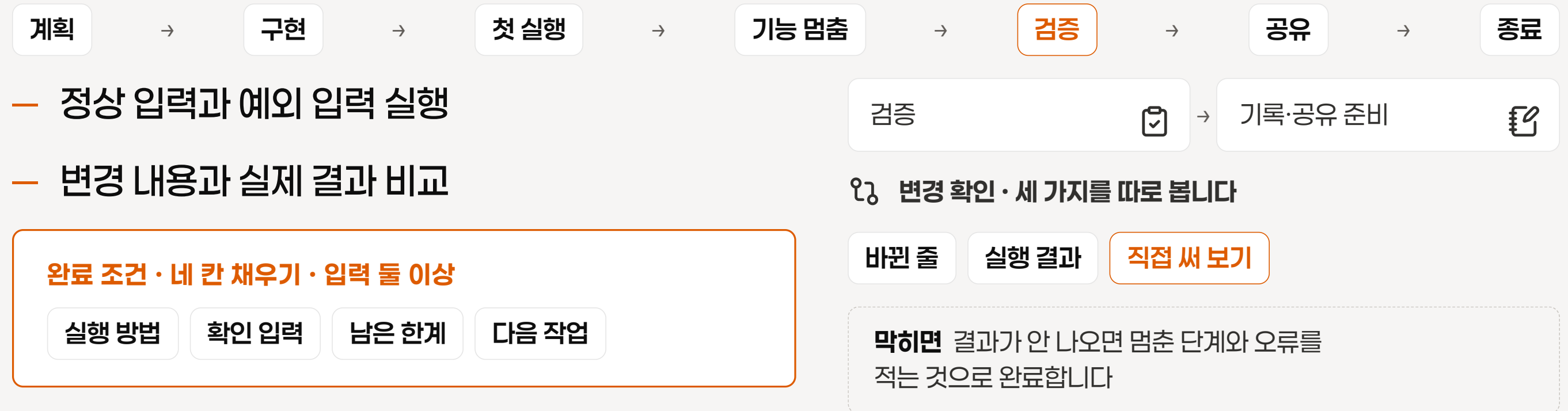
출처: Claude Code 공식 문서 Best practices

AI Agent Guide · SK hynix 사내 교육

자유 실습 3단계 · 검증과 기록

실습 가이드 해당 장

관찰할 것 Claude Code가 완료했다고 말해도 변경 내용과 실제 실행 결과로 판단합니다. 확인하지 못한 입력은 미확인으로 적습니다.



출처: Claude Code 공식 문서 Best practices



완성 여부보다 **확인한 것과 남은 것**을 말합니다

사례 공유 · Q&A · 마무리

결과가 완성되지 않아도 세 질문에 답하면 됩니다. 발표하지 않은 분은 세 질문의 답을 기록 파일에 적습니다.

1 해결하려던 문제는?

2 확인한 결과와 남은 한계는?

3 다음에 적용할 업무는?

현업 적용 전 점검은 부록 참고

오늘 남길 4가지 성과

오늘 만든 결과물보다 오래 남길 것은 이 네 가지 성과입니다.

설명하기



- 각 요소가 하는 일

모델 · API · RAG · Tool · Agent

고르기



- 필요한 만큼의 수준

스크립트 · Workflow · Agent

전달하기



- 목표와 완료 조건

입력 · 출력 · 제약 · 완료 조건

검증하기



- 결과를 근거로

근거 데이터 · 다른 입력 · 실패 상황

LLM에게 맡길 일과 코드·데이터·사람이 책임질 일을 구분하면 작은 업무 도구부터 만들고 개선할 수 있습니다.

부록 · Transformer와 Q·K·V

Attention은 Q·K 비교로 정한 비율대로 V를 모읍니다.

비율은 입력마다 새로 계산되는 값이며 학습된 매개변수와 다릅니다. 원 논문은 encoder-decoder 구조이고, 많은 생성 모델은 decoder 중심입니다.

— Q·K·V 찾는 기준·이름표·내용

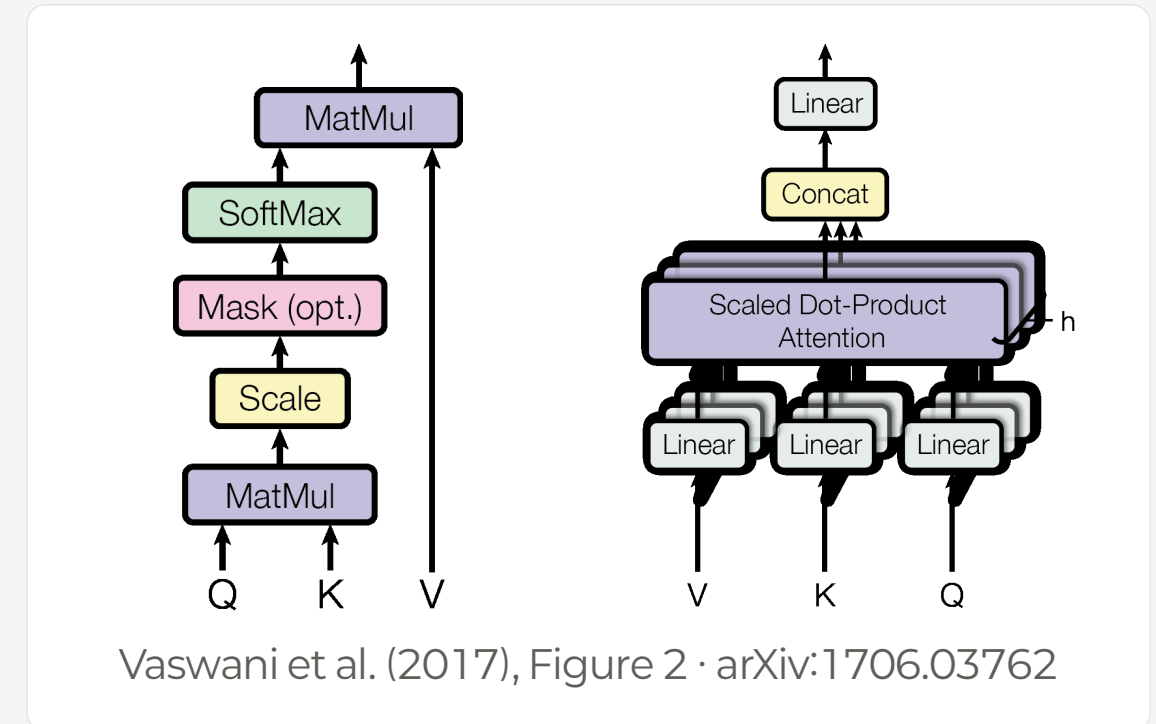
비유 · 한 토큰 표현에서 나온 세 벡터

— 그림 왼쪽: 헤드 하나의 계산

$\text{softmax}(QK^T / \sqrt{d}) \cdot V$

— 그림 오른쪽: 여러 헤드를 나란히

헤드 = Q·K·V 계산 한 벌 · 헤드마다 다른 변환



출처: Vaswani et al., Attention Is All You Need (2017) · arXiv:1706.03762

부록 · RAG 품질 다듬기

RAG 품질은 재학습이 아니라 분할·검색·메타데이터로 다듬습니다.

메타데이터는 답변에서 구분할 조건부터 설계합니다. 답이 틀리면 검색이 틀렸는지, 찾은 근거로 잘못 답했는지부터 나눕니다. 검색 쪽은 재정렬(Reranking), 답변 쪽은 프롬프트 제약을 봅니다.



출처: Lewis et al. 2020 · LangChain Retrieval 가이드

부록 · 프레임워크 선택

배우는 순서가 아니라 문제에 필요한 추상화 수준을 고릅니다.

추상화 수준은 기능이 미리 묶인 정도입니다. 단순 호출 하나면 SDK로 충분하고, LangChain은 모델·검색기·도구를 같은 방식으로 잇는 연결 도구일 뿐 모델을 더 똑똑하게 만들지 않습니다. LangChain Agent와 Deep Agents는 LangGraph 위에서 동작합니다.

짧게 구성



LangChain Agent

- 모델·도구 호출 루프
- create_agent 한 번으로

흐름 직접 설계



LangGraph

- 단계·분기를 직접 정의
- 상태 저장·중단 후 재개

기능 묶음



Deep Agents

- 계획·파일·위임 내장
- 여러 단계의 긴 작업용

공통 실행 기반 · LangGraph 런타임

개념 예시 · 기반 관계, 배우는 순서 아님

출처: LangChain Overview · LangGraph Overview · LangChain Concepts(Agent harnesses: Deep Agents)

부록 · Skill·MCP·Subagent·Hook

넷은 상하위 단계가 아니라 서로 다른 역할입니다.

Skill은 반복 절차를 파일로 적어 두고 필요할 때 불러 쓰는 것이며 모델을 학습시키는 기능이 아닙니다. 네 기능은 필요할 때만 골라 조합하고, 기본 도구로 충분하면 더하지 않습니다. CLAUDE.md 지침은 권한을 강제하지 않으므로 설치·권한은 사내 가이드의 승인된 설정을 따릅니다.

절차 파일 Skill - 반복 절차를 SKILL.md에 - 이름·설명만 보다가 고르면 본문 읽기 예: 노트 정리 절차	외부 연결 MCP 앱 N개·도구 M개를 규약 하나로 N×M → N+M 개념 예시 · 앱·도구 연결 수	작업 분담 Subagent - 별도 맥락에서 하위 작업 - 결과를 주 대화로 반환 예: 예외 입력만 점검	이벤트 처리 Hook - 정해진 시점에 자동 실행 - 명령 실행형·모델 판단형 예: 파일 수정 후 포맷
---	---	--	---

출처: Claude Code 문서 — Skills · MCP · Subagents · Hooks

부록 · 실행 오류 점검

막히면 접속부터 파일 경로까지 한 층씩 좁힙니다.

호환 API라도 모든 옵션이 똑같이 동작하지는 않습니다. 오류 원문을 남기고 한 번에 한 가지만 바꿔 다시 실행합니다.
주소·계정·키는 사내 실행 가이드에서 확인합니다.



출처: vLLM Online Serving 문서 · Claude Code 공식 문서 How Claude Code works

부록 · 현업 적용 전 점검

화면이 뜨는 것과 업무에 쓸 준비는 다릅니다.

오늘 만든 작은 도구를 업무에 쓰기 전에 다섯 가지를 확인합니다. 권한은 프롬프트 문장이 아니라 실행 환경에서 제한합니다. 없는 답을 만들지 않고 실패를 알리는지도 봅니다.



출처: LangChain SQL 안전 지침(SQL Agent 가이드 내) · Anthropic, Building effective agents

부록 · LangGraph 흐름 설계

분기·중단·재개가 필요할 때 흐름을 그래프로 직접 그립니다.

LangGraph는 상태(State)·노드(Node, 처리 단계)·연결(Edge)로 흐름을 정의합니다. create_agent의 도구 호출 루프도 내부는 그래프입니다. 순서가 고정된 작업이면 직접 그리지 않아도 됩니다.



모두 END로 · 개념 예시, 오전 앱 구조 아님

StateGraph 빌드 · 교육용 축약(일부 줄 생략)



```
g = StateGraph(State)
g.add_node("classify", classify)
g.add_edge(START, "classify")
g.add_conditional_edges("classify", route)
app = g.compile(checkpointer=...)
app.invoke({"messages": [...]}, config)
```

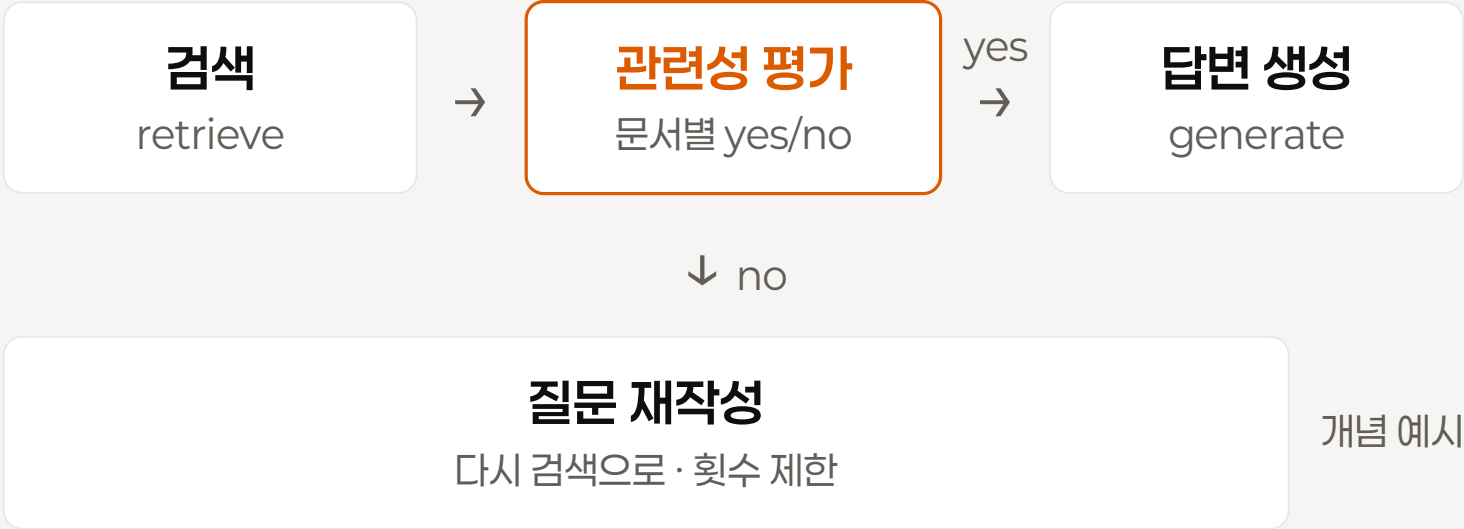
출처: LangGraph Overview

AI Agent Guide · SK hynix 사내 교육

부록 · Agentic RAG

검색 결과를 평가해 부족하면 질문을 고쳐 다시 찾습니다.

앞의 RAG 구간에서 본 '검색이 틀렸나' 확인을 그래프에 넣은 구조입니다. 모델이 문서마다 관련성을 yes/no로 매기고, 그 결과로 경로가 갈립니다. 판정마다 호출이 늘어 재시도 횟수를 제한합니다.



관련성 판정 스키마 · 교육용 축약(일부 줄 생략)

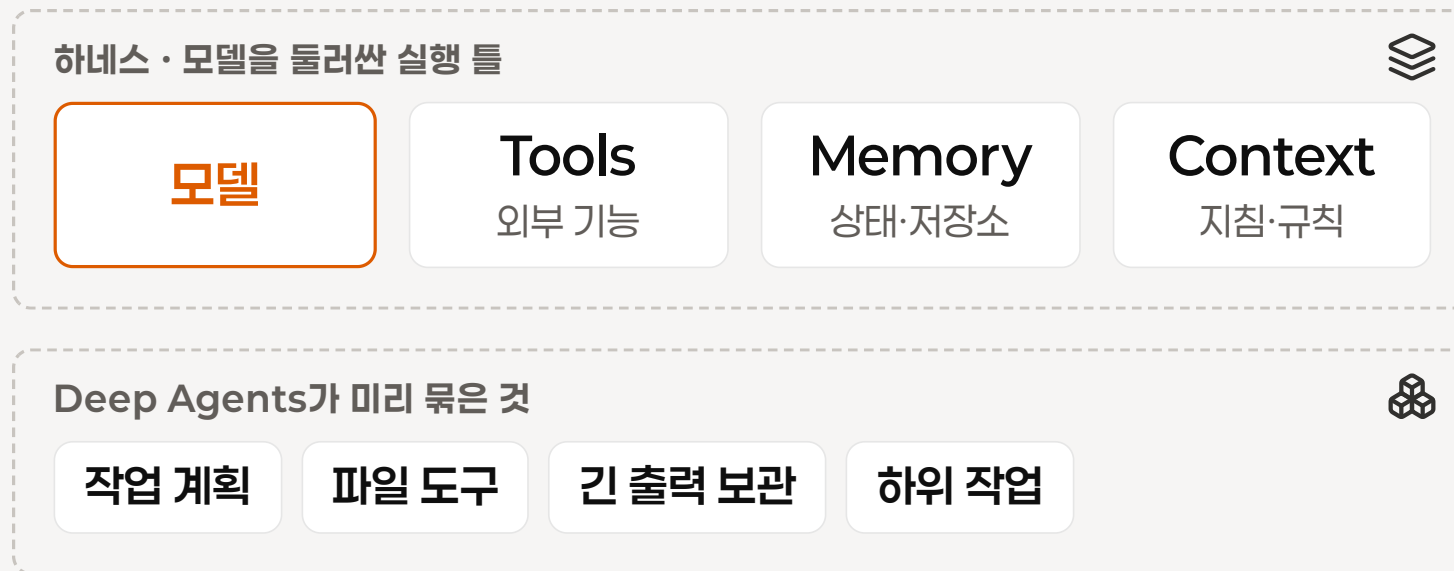
```
class GradeDocuments(BaseModel):
    score: Literal["yes", "no"]
grader = model.with_structured_output(
    GradeDocuments)
grade = grader.invoke(prompt).score
```

출처: LangGraph Agentic RAG 문서 · Lewis et al. 2020

부록 · 하네스와 Deep Agents

하네스는 모델에 도구·기억·맥락을 묶는 실행 틀입니다.

Deep Agents는 LangGraph 위에 계획·파일·위임 등을 미리 묶은 하네스입니다. Claude Code도 구현은 다르지만 같은 역할의 실행 틀입니다. 짧은 도구 호출이면 create_agent로 충분합니다.



개념 예시

create_deep_agent 인자 · 교육용 축약(값 생략)



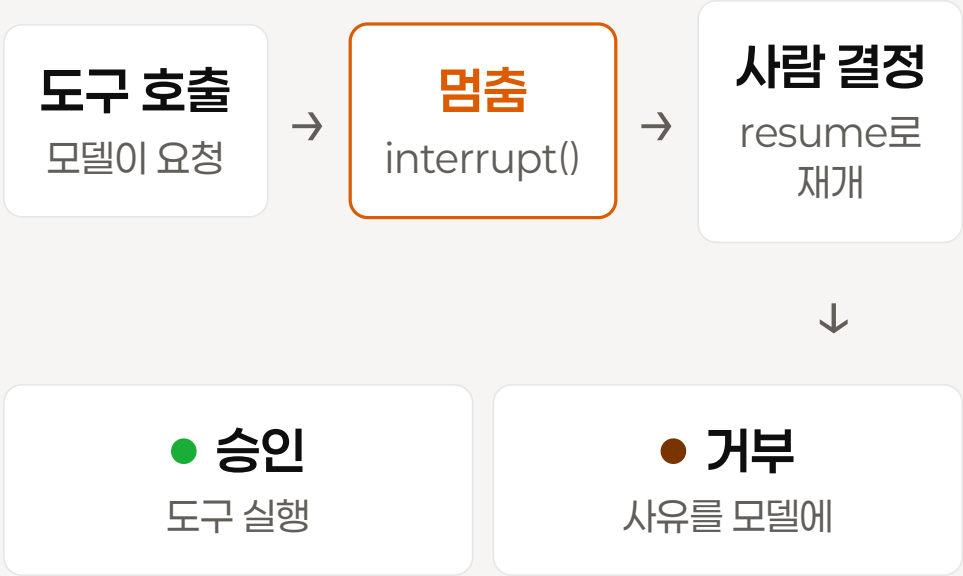
```
agent = create_deep_agent(  
    model=..., tools=[...],  
    subagents=[...], skills=[...],  
    backend=...,  
    interrupt_on={...},  
)
```

출처: LangChain Concepts(Agent harnesses: Deep Agents)

부록 · 사람 승인(HITL)

되돌리기 어려운 도구 실행 전에 멈추고 사람의 결정으로 재개합니다.

interrupt()로 멈추고 Command(resume)로 사람의 결정을 받아 이어갑니다. 멈춘 상태를 저장할 체크포인트가 필요합니다. CLAUDE.md나 프롬프트의 '하지 마' 문장은 이것을 대신하지 못합니다.



지정한 도구만 멈춤 · 결과는 다시 모델로 · 개념 예시

셸 명령 실행 승인이 필요합니다

도구 · execute

```
{  
  "command": "python3 --version"  
}
```

거부 사유 (선택)

승인

거부

사내 배포 패키지 예시 화면의 승인 카드 · 실행할 명령과 승인·거부 버튼 · 수강생 실습 화면 아님

출처: LangGraph Human-in-the-loop 문서